SIGMA

Arquitectura Offline First
React Native + SQLite + Express + MySQL

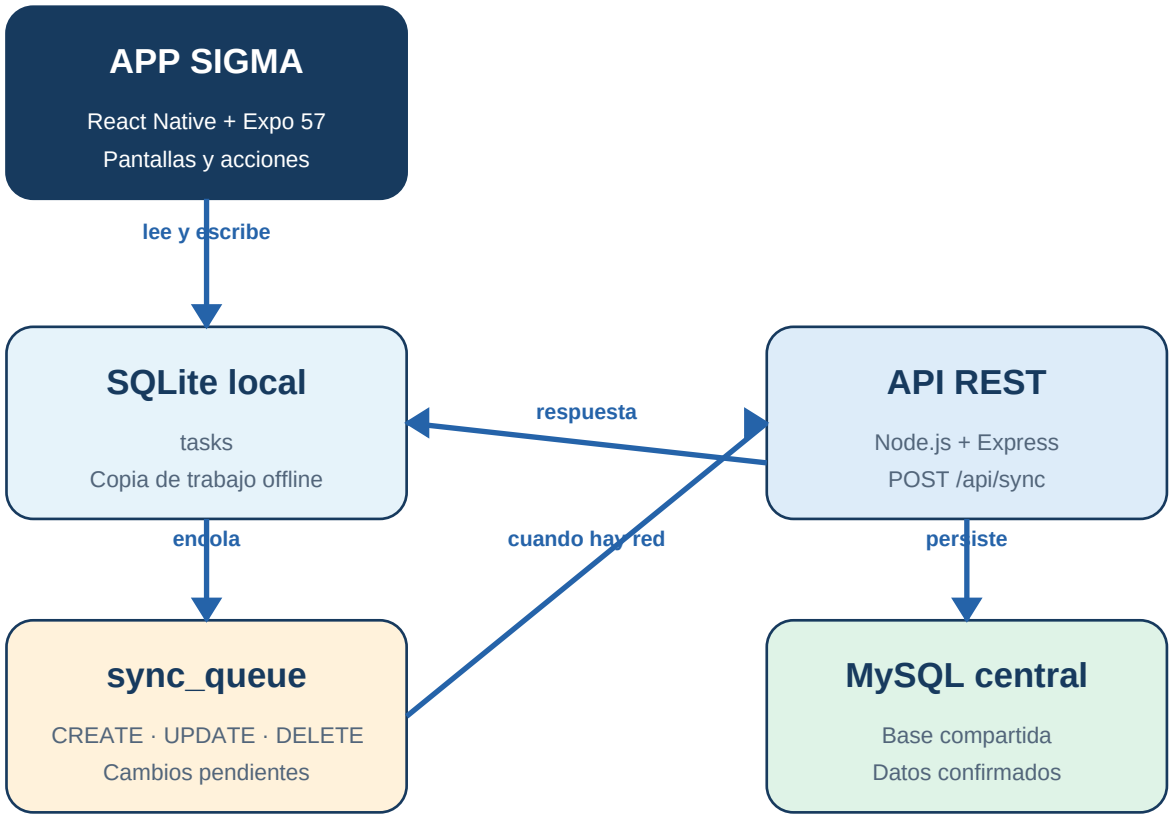
Documento de apoyo para la clase de Desarrollo para Dispositivos Móviles

Versión didáctica: backend Node.js básico, persistencia local real y sincronización visible.

Arquitectura de la nueva opción 2

La aplicación usa SQLite como fuente local de datos. El usuario puede trabajar sin conexión; cada operación se registra en la cola y luego se replica a la API cuando vuelve Internet.

El celular trabaja primero en SQLite; el servidor confirma después.

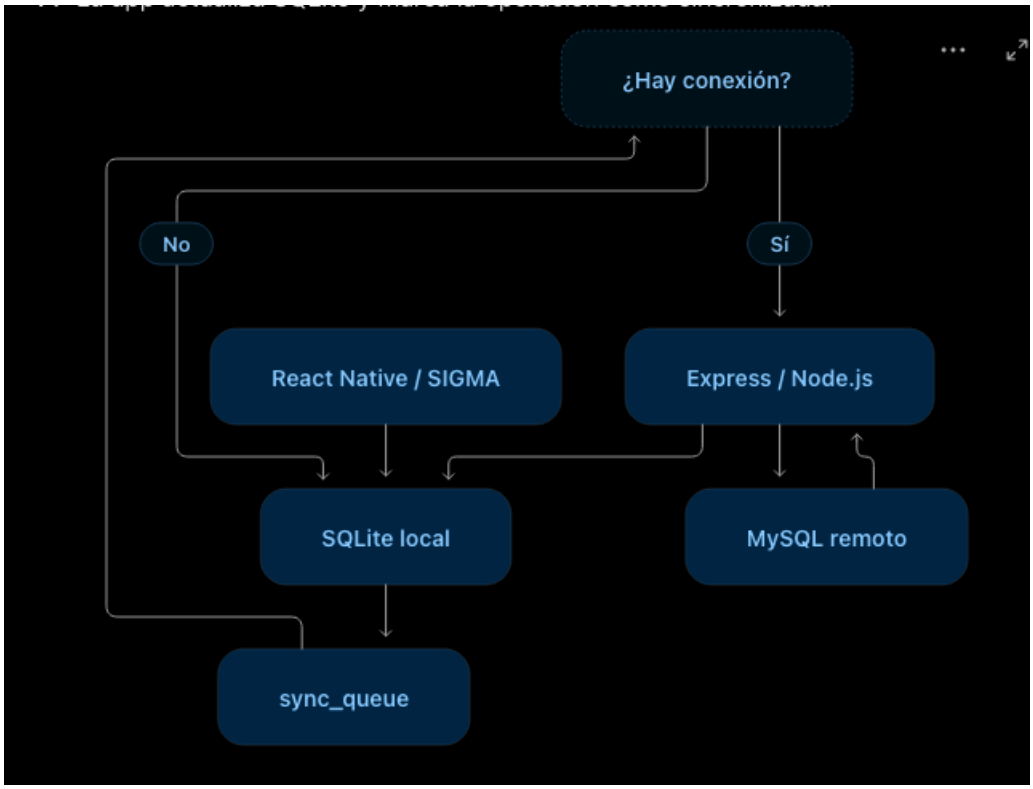


Lectura del diagrama

APP SIGMA consulta y modifica SQLite. Cada modificación también genera una entrada en **sync_queue**. Con conexión disponible, la cola viaja a Express mediante `POST /api/sync`; Express la guarda en MySQL y devuelve una copia central actualizada. La app aplica esa respuesta sin sobrescribir cambios que todavía estén pendientes.

Diagrama de decisión de sincronización

Este diagrama complementa la arquitectura anterior. Señala la decisión que toma la aplicación al detectar conectividad: sin red, React Native continúa trabajando con SQLite y acumula operaciones en `sync_queue`; con red, Express recibe esos cambios y los confirma en MySQL.



El punto clave es que la falta de conexión no bloquea al usuario. La sincronización se posterga, pero la operación ya quedó almacenada de forma durable en el teléfono.

1. Decisión de arquitectura

La propuesta conserva el objetivo central de SIGMA: registrar tareas de mantenimiento en un celular aun cuando no existe conexión. La simplificación se realiza en el servidor: se sustituye NestJS por Express y PostgreSQL por MySQL, sin perder SQLite local ni la cola de sincronización.

La app no espera al servidor para responder. Cada acción del usuario se confirma primero en el teléfono; luego se replica al sistema central.

Capa	Tecnología	Responsabilidad
Aplicación móvil	React Native + Expo 57	Pantallas de tareas, estado visual y acciones del usuario.
Persistencia local	expo-sqlite	Copia local de tareas y operaciones pendientes. Funciona sin red.
Conectividad	NetInfo	Indica si es razonable iniciar una sincronización.
API	Node.js + Express	Recibe HTTP, valida datos y coordina la transacción remota.
Base central	MySQL 8	Información compartida y persistente de todos los dispositivos.

Por qué Express y MySQL

El objetivo de esta etapa es enseñar con claridad cómo se conectan una API REST, SQL y una app offline-first. Express evita la estructura adicional de NestJS y `mysql2` muestra consultas parametrizadas de MySQL sin introducir un ORM. La solución sigue siendo extensible: más adelante se pueden sumar autenticación, módulos, validaciones y una capa ORM.

2. Datos locales, remotos y pendientes

SQLite tiene dos tablas. La primera contiene el estado de trabajo de las tareas; la segunda registra qué cambios todavía deben viajar. Esa separación evita perder acciones cuando la conexión falla.

Tabla SQLite	Contenido	Uso
tasks	id, título, estado, fechas, estado de sincronización	Permite cargar la lista al instante, aun sin Internet.
sync_queue	CREATE, UPDATE o DELETE más un JSON de la tarea	Es una cola durable de cambios pendientes.

Estados de sincronización

Estado	Significado
PENDIENTE	El cambio ya se guardó en SQLite, pero todavía no fue confirmado por MySQL.
SINCRONIZADA	El backend respondió correctamente y no queda ninguna operación pendiente para esa tarea.
ERROR	Falló el intento de envío. La operación permanece en la cola para que pueda reintentarse.

MySQL conserva la tabla `tasks` con los mismos identificadores. El campo `deleted_at` permite eliminación lógica: el servidor conoce que una tarea fue eliminada y puede replicarlo a otros dispositivos.

3. Flujo de trabajo offline-first

El flujo se diseña para que el usuario nunca dependa de que la red responda antes de continuar.

Paso	Qué sucede
1. Apertura	La app consulta SQLite y muestra las tareas locales. No realiza una espera obligatoria por la API.
2. Cambio local	Al crear, finalizar o eliminar una tarea, se actualiza `tasks` y se agrega una fila a `sync_queue` dentro de una transacción SQLite.
3. Sin señal	La pantalla sigue funcionando. Las operaciones quedan PENDIENTE y se conservan después de cerrar la app.
4. Recuperación de red	NetInfo detecta conectividad. En este ejemplo el docente o alumno presiona Sincronizar ahora para observar el proceso.
5. Envío	La app manda todas las operaciones pendientes en `POST /api/sync`.
6. Confirmación	Express procesa la lista en una transacción MySQL y devuelve los identificadores confirmados y la copia central de tareas.
7. Actualización local	La app elimina de `sync_queue` las operaciones confirmadas, marca las tareas como SINCRONIZADA y aplica los cambios remotos sin pisar cambios locales pendientes.

Ejemplo de una operación en cola

```
{ "queueId": 7, "type": "UPDATE", "task": { "id": "...", "title": "Revisar impresora",
"status": "FINALIZADA", "updatedAt": "..." } }
```

4. API y reglas de sincronización

Método	Ruta	Propósito
GET	/api/health	Comprueba que Express y MySQL estén disponibles.
GET	/api/tasks	Devuelve la copia central, incluso eliminaciones lógicas.
POST	/api/tasks	Crea una tarea desde una herramienta externa.
PATCH	/api/tasks/:id	Actualiza una tarea conocida.
DELETE	/api/tasks/:id	Marca una tarea como eliminada.
POST	/api/sync	Recibe la cola de un celular y devuelve acuses y tareas remotas.

Consistencia y conflictos

Cada tarea tiene `updated_at`. En esta versión, el cambio con fecha más reciente prevalece. Es una regla simple y razonable para ejercicios individuales, pero no basta para trabajo colaborativo crítico. Una evolución de producción agregaría versión incremental, usuario responsable, historial de cambios y una pantalla de resolución de conflictos.

La operación de sincronización se procesa dentro de una transacción MySQL: o se confirman todas las operaciones recibidas, o ninguna. Esto evita una sincronización a medias en el servidor.

5. Qué se aprende y cómo evoluciona

Nivel	Conceptos que se trabajan	Posible siguiente paso
Ejemplo actual	SQLite, cola durable, REST, Express, MySQL, transacciones y conectividad.	Sincronización automática al recuperar señal.
Proyecto SIGMA	Usuarios, órdenes de trabajo, activos, sectores y roles.	JWT, validación de permisos y auditoría.
Trabajo de campo	Fotos, QR, GPS y evidencia de intervención.	Cola de archivos, almacenamiento de objetos y reintentos por archivo.
Producto colaborativo	Varios técnicos modificando datos.	Versionado, conflictos explícitos y sincronización incremental.

Ventajas

La solución es fácil de ejecutar localmente, muestra SQL real, tolera falta de conectividad y mantiene una separación clara entre los datos del teléfono y los datos centrales.

Límites conscientes

No incluye autenticación, seguridad avanzada, adjuntos ni estrategia de conflictos compleja. Son decisiones intencionales para mantener el primer ejemplo enfocado y comprensible.

Conclusión

Para SIGMA, SQLite local + cola de cambios + Express + MySQL ofrece un punto de equilibrio adecuado: enseña una estrategia offline-first completa sin ocultar las piezas fundamentales detrás de un framework complejo.